SpartanAssist: A Multi-Agent AI IT Help Desk

BUS4 119A MIS Practicum · Capstone · Team Auto-Mates (Nhu Vo, Liz Pham, Sri Sudula, Kimberly Gomez, Yuvraj Singh, Jorge Perez) · Nov 12, 2026

Executive summary

SpartanAssist is a LangGraph multi-agent help desk that resolved 70% of 20 labeled campus IT scenarios on its own and sent the other 30% to a person or an out-of-scope reply, as policy says it should. On a held-out set it got 90% of categories and routes right.

Six nodes share one state: a Privacy Guard, then the Intake, Knowledge (RAG), Workflow and Escalation agents, then a Responder. Automations run through a real Model Context Protocol (MCP) server. VS Code + GitHub Copilot connects to that same server, so IT staff can read and update the tickets the agents create.

Headline result	Value
Category accuracy (20 / held-out 10)	100% / 90%
Routing accuracy (20 / held-out 10)	100% / 90%
Retrieval hit@2	100%
Answers citing a retrieved KB article	100%
Auto-resolved vs escalated	70% vs 25% (sample notebook: 0% vs 100%)
Latency p95, offline mode	64 ms (LLM mode to be re-measured)

The most important finding is a failure. One held-out account-takeover message was classified as a login issue and auto-resolved. We treat security false negatives as a release blocker and list the fix in Testing.

Use case and success metrics

Our product is a 24/7 first line of campus IT support. It fixes the repetitive tickets on its own and hands everything else to the right team with the context already collected.

Problem. Campus help desks take a flood of repetitive tickets: Wi-Fi, passwords and lockouts, MFA, software installs, printing, Canvas. Volume peaks at semester start and during finals, exactly when students have the least time to wait. Walk-in hours are fixed, but deadlines come at midnight.

Users and jobs-to-be-done

Persona	Job to be done	Pain today
Student (primary)	"Get me back into Canvas before my deadline"	Email queue, re-explaining the problem
Faculty / staff	"Fix my classroom Wi-Fi or VPN fast"	Phone holds during class
Help desk technician	"Only see tickets that need me, with context"	Triage time, repeated questions
IT manager (product owner)	"Deflect tier-1 volume without hurting trust"	No data on what is automatable

In scope: Wi-Fi/VPN, password reset, account unlock, MFA, software licensing, laptop issues, email, printing, Canvas, phishing reports. **Out of scope:** non-IT questions (parking, dining, essays). The assistant declines these instead of guessing.

Priority model: P1 = exam or class in progress, security incident, or outage hitting many users; P2 = blocked with a deadline; P3 = degraded, workaround exists; P4 = how-to question.

Success metrics

Metric	Target	Why it matters
Category accuracy	at least 90%	Wrong category means wrong answer and wrong team
Routing accuracy (resolve vs escalate)	at least 85%	Core product promise
Retrieval hit@2	at least 90%	RAG grounding quality
Grounded answers (cite a retrieved KB article)	100%	Hallucination control

Metric	Target	Why it matters
Security false negatives	0	Missed compromises are the worst failure
Latency p95	under 8 s	The sample notebook averaged 16.5 s
CSAT (pilot survey, share scoring 4 or 5)	at least 80%	User satisfaction requirement
Auto-resolution rate	50 to 70%	Deflection without over-automation

Agent design

Each agent owns one decision and writes it to a shared `HelpDeskState`. Routing between agents is decided by explicit rules, so every hand-off can be explained and tested.

Node	Owns the decision	Reads	Writes	Tools
Privacy Guard	What text is safe to process	raw message	redacted query, PII types found	regex redaction (IDs, email, phone, passwords)
Intake Agent	What is this, how urgent, what action?	redacted query	category, P1-P4, confidence, workflow, wants-human	Gemini structured output (Pydantic schema), or keyword + KB-vote classifier offline
Knowledge Agent	What does our KB say?	query, category	top-2 chunks, answer, citations, grounded flag	ChromaDB vector search, similarity threshold, citation check
Workflow Agent	Can we fix it automatically?	workflow, user	tool result, ticket id	MCP tool server: password reset, unlock, diagnostics, service status, license check, tickets

Node	Owns the decision	Reads	Writes	Tools
Escalation Agent	Does a person need this, and who?	full state	reasons, team, channel, SLA, handoff note	rules + optional LLM polish; updates the ticket
Responder	What does the student see?	full state	final message, status	none

Routing rules (LangGraph conditional edges)

1. Privacy Guard, then Intake, always.
2. Intake: clearly off-topic goes straight to the Responder, with no retrieval spent. Everything else goes to Knowledge.
3. Knowledge: P1 or security goes to Escalation (fast lane, with the KB's do-this-now steps attached). An action is needed: go to Workflow. Otherwise go to the Responder.
4. Workflow: any escalation rule fires (automation failed, physical repair, user asked for a person, low confidence): go to Escalation. Otherwise go to the Responder.
5. Escalation always ends at the Responder, so the student gets one message with the steps, what was done, and who has the case.

Why this split. Separating concerns means the Knowledge Agent never needs write access to identity systems. Only the Workflow Agent touches tools, and only the tools the MCP server exposes. That is the same least-privilege pattern enterprise platforms use, and it lets each agent scale or be swapped on its own (for example, Gemini for Intake and a cheaper model for the Responder).

UX design

The interface is one chat window built around a single promise: in every reply the student can see what happened, why to trust it, and what comes next. The wireframes are in the team's UX Wireframes canvas (folder 03 of the package), and the notebook's Gradio chat (section 15) implements a simplified version of screens 1 and 2 (quick-start examples and a status badge).

Design principles (clarity and ease of use)

1. **Lead with the fix, not the explanation.** Numbered steps first, at most 6, one action each.

2. **Show your source.** Every answer carries a KB citation chip. No chip means no answer, and the case goes to a person.
3. **Say what you did.** Automations appear as a separate "Done for you" card, so students don't redo them.
4. **Always a way out.** A "Talk to a person" button on every reply, and the words "real person" in the message trigger it too.
5. **Status in words and color.** Resolved (green), With a person (amber), Out of scope (gray). Color is never the only signal.
6. **Privacy feedback.** If the student types a password or ID, the reply says it was hidden and warns them not to share it.
7. **Accessible by default.** Plain language at roughly 8th-grade reading level, keyboard-only flow, WCAG AA contrast, works at 375 px wide on mobile.

Screens

#	Screen	Key elements	Principle shown
1	Start	Greeting, 4 quick-start chips (Wi-Fi, Password, Software, Printing), message box, service status strip	Ease of use: no blank page
2	Resolved answer	Steps, KB source chip, Done-for-you card, ticket number, "Did this fix it?" Yes / No	1, 2, 3
3	Handed to a person	Amber banner, team, channel and response target, do-this-now steps, ticket number	4, 5
4	Staff view (VS Code + MCP)	Copilot Chat listing escalated P1 tickets with handoff notes	Technician never re-asks

Usability check. Before the demo, Jorge runs the 5-person hallway test in the survey file: each person completes 3 tasks (reset a password, fix Wi-Fi, report phishing). Measure task success, time on task, and CSAT.

System implementation

One Colab notebook builds and runs the system end to end: `SpartanAssist_AutoMates_Capstone.ipynb`, with 16 sections and outputs saved. Run modes are `gemini` (the default when a `GOOGLE_API_KEY` secret is set; gemini-2.5-flash plus gemini-embedding-001), `openai` (gpt-4o-mini), or `offline` (local TF-IDF plus rule-based agents, so the demo survives a missing key or a rate limit).

Layer	Choice	Detail
Orchestration	LangGraph `StateGraph`	6 nodes, 3 conditional edges, typed shared state
LLM	Gemini 2.5 Flash, temperature 0	Structured output for Intake; grounded prompt for Knowledge
Knowledge base	11 runbook articles, 24 chunks	Split on section headings so retrieval returns one procedure, not a whole article
Vector store	ChromaDB, cosine	We pass our own vectors, so the code is identical in every mode
Workflow tools	`helpdesk_tools.py` + SQLite ticket table	Password reset, unlock, network diagnostics, service status, license check, create / update / list tickets
MCP	`helpdesk_mcp_server.py` (official `mcp` SDK, FastMCP, stdio)	7 tools + a `kb://articles/{id}` resource
MCP client	VS Code + GitHub Copilot via `.vscode/mcp.json`	Also lists the GitHub and Atlassian Rovo remote servers
UI	Gradio `ChatInterface` (optional)	Shareable link for the live demo

RAG best practices we applied

- **Grounding:** chunks below a similarity floor (0.55 in Gemini mode, 0.10 for TF-IDF) are dropped. No surviving chunk means no answer and the case goes to a person.
- **Citation check:** the model must cite `[KB-…]` ids. The answer is discarded if it cites nothing, or cites an article that wasn't retrieved.

- **Context precision:** heading-level chunks plus a small boost for the Intake category and for matching section titles. The sample notebook returned the VPN guide for a Wi-Fi question; ours doesn't.
- **Privacy:** PII is redacted *before* embedding, prompting or logging. Offline mode keeps all text on the machine; production would use a campus-approved Gemini tenant with no training on our data.

Workflow automation. We picked the processes that are repeatable and low-risk: reset links, unlocks, diagnostics, status checks and license checks. Every conversation logs a ticket, which gives an audit trail and makes the metrics queryable. Anything that changes privileges (disabled accounts, MFA re-enrollment) stays with a human.

MCP vs traditional APIs. With REST, each client hard-codes each endpoint. With MCP, any client calls `tools/list`, gets names, descriptions and JSON schemas at runtime, and calls them through one protocol. In section 14, the notebook connected to our server, discovered all 7 tools and created a ticket without importing any server code. VS Code does the same through `mcp.json`. Moving to production means swapping a tool's body for a Jira Service Management call through Atlassian's Rovo MCP server; the agents don't change.

Testing and validation

SpartanAssist met every target on the 20 labeled scenarios and scored 90% on 10 held-out phrasings. The held-out run surfaced one security miss that must be fixed before launch. All results come from the run on Nov 12, 2026 in offline mode; raw rows are in `test_results.csv` and `heldout_results.csv`.

Method. The team wrote the expected category, severity, KB article and outcome for each scenario *before* running the system. The 20 scenarios cover all 9 categories, all 4 priority levels, every route, a PII case and an off-topic case. Because we also used those 20 to debug the offline classifier, we wrote 10 more after tuning and never changed code based on them.

Metric	Target	Main set (20)	Held-out (10)
Category accuracy	at least 90%	100%	90%
Severity exact / within one level	at least 90% (within one)	85% / 100%	not labeled

Metric	Target	Main set (20)	Held-out (10)
Routing accuracy	at least 85%	100%	90%
Retrieval hit@2	at least 90%	100%	not labeled
Grounded answers	100%	100%	not labeled
Outcomes: resolved / person / out of scope	policy	14 / 5 / 1	7 / 2 / 1
Latency p50 / p95	p95 under 8 s	8.6 ms / 64.1 ms	not measured
CSAT proxy (1-5 rubric)	at least 4.0	4.8	not measured

Failures and fixes

ID	Message	What happened	Fix
H09	"someone is sending emails from my account asking for gift cards"	Read as a login problem and auto-resolved. **A missed account takeover.**	Add takeover cues to the security rules; add 10 security paraphrases to the test set; block release on any security false negative
H05	"Excel keeps crashing when I open big files"	Right category, but no KB article covers crashes, so the answer came from weakly related chunks	Write an Office-crash article; raise the offline similarity floor
Main set	3 severity labels one level off (for example, P3 labeled, P4 predicted)	Within tolerance; no routing impact	Tune once LLM mode is measured

Caveats. (1) Latency is offline mode; each Gemini call adds about 1 to 3 s. Re-run section 13 with a key before the demo and update this table. (2) The CSAT number is a rubric proxy. Real CSAT comes from the 5-to-10-person pilot survey in folder 05. (3) The 70% auto-resolution rate reflects our scenario mix, not live campus traffic.

Against the sample notebook: the sample sent 4 of 4 demos to a person, averaged 16.5 s, and reported only the model's own confidence. We report accuracy against labels, resolve 70% automatically, and escalate only when a written policy rule fires.

Industry landscape and positioning

SpartanAssist is a lightweight, open prototype of what enterprise vendors now sell as agentic service desks. It uses the same building blocks (enterprise search, workflow agents, MCP), sized for one campus and built for $0 in licenses.

Vendor	What it offers	What we borrowed	Where we differ
Glean	Permission-aware enterprise search; Glean Agents with 30+ prebuilt agents, including help desk automation	Search-first grounding: answer from company knowledge, cite it	Glean is strongest at search; full ticket lifecycle usually sits in an ITSM tool. We include a ticket workflow
ServiceNow + Moveworks	ServiceNow closed its Moveworks deal in Dec 2025 ($2.85B announced): a conversational front end on top of ITSM workflows	Front door (chat) separated from back end (workflow agents)	Enterprise licensing and implementation cost; we are a notebook a campus team can read end to end
Atlassian Rovo MCP server	Exposes Jira, Confluence and Compass as MCP tools; respects each user's permissions	Our production path for tickets	We prototype the same tool contract with a local MCP server
GitHub MCP server	Remote MCP server for repos, issues and PRs, usable from VS Code	KB-as-code: runbooks in a repo, KB gaps filed as issues	None. Used as is

Our positioning. For a university IT office that wants to pilot agentic support before buying a platform, SpartanAssist shows the value (deflection, faster P1 routing, a full audit trail) on its own data, with open components. Because tools sit behind MCP, the pilot

moves to Jira, ServiceNow or Glean later without rewriting the agents.

ServiceNow says its own agents now resolve 90% of internal IT requests autonomously, but it hasn't published the baseline (UC Today). That's a reminder to judge any vendor's deflection number, ours included, against a labeled test set.

Trade-offs and lessons learned

Our biggest lesson was that explicit rules plus an LLM beat an LLM alone for decisions people must be able to audit.

Decision	We chose	We gave up	Why
Escalation logic	Written rules (P1, security, low confidence, not grounded, failed automation, repair, user asked)	Letting the LLM decide	Testable, explainable to IT managers, no prompt drift
Retrieval	Similarity floor + citation check	Some answers we could have given	A wrong confident answer costs more trust than a hand-off
Privacy	Redact before any model sees text	Some context (the model never sees IDs)	PII never reaches logs, embeddings or vendors
Tools	MCP server instead of direct API calls	A little latency and setup	One contract for agents, VS Code and future vendors
Runtime	Offline mode alongside Gemini	Answer fluency in offline mode	The live demo can't be killed by a rate limit or a missing key
Model	Gemini 2.5 Flash	Some reasoning depth versus larger models	Speed and cost fit tier-1 support

What we learned

- **Test sets keep you honest.** The sample notebook looked like it worked because it reported the model's own confidence. Labeled tests showed our real gaps, and the held-out set caught one the main set hid.
- **Thresholds matter more than k.** Dropping weak matches fixed off-topic retrieval and created an honest "I don't know" path.
- **Security needs its own bar.** Overall accuracy was 90%, but a single security miss is unacceptable. That needs a separate metric.
- **MCP paid off twice.** Once we built the tool server for the agents, the staff-side VS Code experience cost no extra code.

Next steps

- [ ] Fix the H09 security rules (owner: Liz); add 10 security test cases and re-run the held-out set (owner: Jorge)
- [ ] Re-run the test suite in Gemini mode and update latency (owner: Nhu)
- [ ] Run the 5-to-10-person pilot survey for real CSAT (owner: Kimberly)
- [ ] Write the missing Office-crash KB article (owner: Sri)
- [ ] Rehearse the 10-minute demo with the VS Code MCP segment (owner: Yuvraj)
- [ ] Production: connect tickets to Jira Service Management via the Atlassian Rovo MCP server; replace the simulated IdP with the campus identity provider

Team and submission contents

Member	Role	Capstone ownership
Nhu Vo	Tech Lead	LangGraph architecture, RAG, MCP server, test harness
Liz Pham	Developer	Agent prompts and rules, workflow tools, Gradio UI
Sri Sudula	Project Manager / Coordinator	Timeline, knowledge base content, submission package
Kimberly Gomez	Business Lead	Use case, success metrics, industry research, pilot survey
Yuvraj Singh	Presentation Lead	Demo deck, live demo script, UX wireframes

Member	Role	Capstone ownership
Jorge Perez	QA & Testing Lead	Labeled test scenarios, held-out evaluation, security test cases, usability hallway test

Package `AutoMates_SpartanAssist_Capstone.zip`

Folder	Contents
01_Report	This report (export to PDF or Word)
02_Presentation	Demo slide deck (export to PDF or PowerPoint)
03_UX_Wireframes	Wireframes and mockups for 4 screens
04_Notebook_and_Code	Executed notebook, `helpdesk_tools.py`, `helpdesk_mcp_server.py`, `.vscode/mcp.json`, MCP setup guide
05_Testing_and_Validation	Test plan and results, `test_results.csv`, `heldout_results.csv`, `metrics.json`, dashboard PNG, pilot survey
06_Knowledge_Base	11 simulated KB articles (Markdown)

The knowledge base is simulated and modeled on typical university IT services. It is not official SJSU IT guidance.

Sources

- Capstone Project instructions, BUS4 119A (course handout)
- Sample notebook: AI IT Help Desk Copilot (course reference)
- Glean expands horizontal agent platform, Glean press release, May 20, 2025
- ServiceNow completes acquisition of Moveworks, Business Wire, Dec 15, 2025
- ServiceNow buys AI agent company Moveworks for $2.85 billion, Fortune, Mar 10, 2025
- ServiceNow Acquires Moveworks, Strengthens Employee Service Automation Position, UC Today, Dec 16, 2025
- Announcing GA of the Atlassian Rovo MCP server, Atlassian Community
- Set up the GitHub MCP server, GitHub Docs